

Assignment # 2

Title:

Recursion Methods

Submitted to :

- **Instructor :** Sir Atif Saeed
- **Course :** Design & Analysis of Algorithms(CSC-301)

Submitted by:

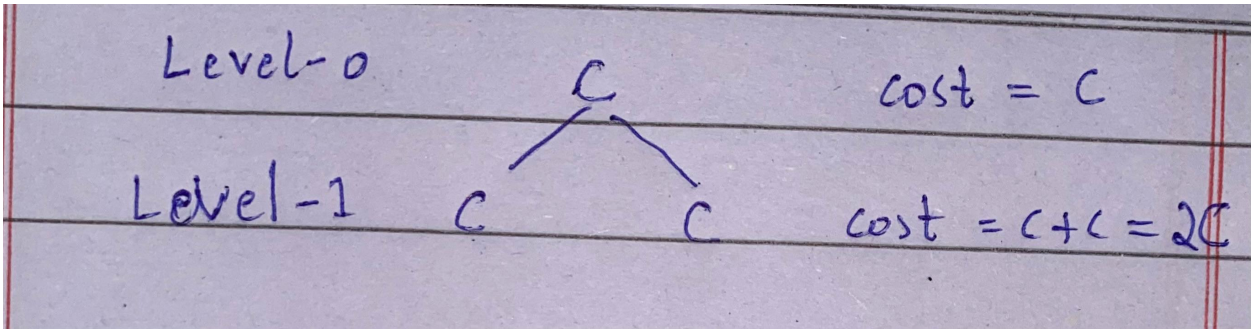
- **Name :** Sannan Ahmad
- **Reg #** (SP25-BSE-118)
- **Section** A

Date of Submission:

27 October , 2026

Tree Method

Rewriting the recurrence: T(n) = 2T(n/2) + C



Level 0: cost = C

Level 1: cost = 2C

Level 0 < Level 1

The cost increases at every level (C < 2C < 4C < …), so the leaves dominate the total cost.

Total Cost = $\Theta(n^{\log_b a})$

a = 2, b = 2

Total Cost = $\Theta(n^{\log_2 2}) = \Theta(n)$

T(n) = Θ(n)

Master Theorem

T(n) = 2T(n/2) + C

a = 2, b = 2, f(n) = C

$T(n) = n^{\log_2 2}$ [U(n)]

T(n) = n · [U(n)]

U(n) depends on R(n)

$R(n) = f(n) / n^{\log_b a} = C / n^1 = C \cdot n^{-1}$

Since $R(n) = \Theta(n^r)$, r = −1 < 0

U(n) = Θ(1)

$T(n) = n \cdot U(n)$

$T(n) = n \cdot (1)$

$T(n) = \Theta(n)$

Substitution Method

$T(n) = 2T(n/2) + C$

$T(n) = 2T(n/2) + C \quad \rightarrow \quad (1)$

$T(n/2) = 2T(n/4) + C \quad \rightarrow \quad (2)$

$T(n/4) = 2T(n/8) + C \quad \rightarrow \quad (3)$

Using eq. (1) and eq. (2):

$T(n) = 2[2T(n/4) + C] + C$

$T(n) = 2^2T(n/4) + 3C$

Using eq. (3):

$T(n) = 2^2[2T(n/8) + C] + 3C$

$T(n) = 2^3T(n/8) + 7C$

Continuing k times:

$T(n) = 2^kT(n/2^k) + (2^k - 1)C$

Base case:

$n/2^k = 1$

$n = 2^k$

$T(1) = 1$

$T(n) = n \cdot T(1) + (n - 1)C$

$T(n) = n(1) + (n - 1)C$

$T(n) = \Theta(n)$

FIN !